

Programmation Web 1

Node.js

Table des matières

Introduction à Node.js.....	3
Votre premier programme node.js.....	3
Saisir de l'information.....	3
Lire un fichier texte.....	4
Écrire un fichier texte.....	5
Serveur Web.....	5
Serveur Web simple.....	5
Routage.....	6
Route avec paramètre.....	7
Fichier HTML.....	8
Fichiers d'images.....	9
Modules et NPM.....	10
Créer un module.....	10
Importation d'un module.....	10
Création d'une classe.....	10
Utilisation d'une classe.....	11
Gestionnaire de paquets NPM.....	11
Services Web avec Express.....	12
Qu'est-ce qu'un service REST?.....	12
Installation d'Express.....	13
Serveur Web.....	13
Serveur Web de fichier.....	14
Serveur Web avec routage.....	14
Route avec paramètres.....	15
Serveur Web répondant en JSON.....	16
Serveur Web recevant du JSON.....	17
nodemon.....	18
Mongoose.....	18
Installation.....	18
Connexion.....	18

Document écrit par Stéphane Gill

Mars 2023



Ce document est soumis à la licence Creative Commons Attribution 3.0 Canada. Permission vous est donnée de copier, modifier et redistribuer des copies de ce document, dans les conditions fixées par la licence, tant que cette note apparaît clairement.

Ajouter un enregistrement.....	19
Lire les enregistrements.....	20
Service Web REST.....	21
Session.....	23
Sécurité et authentification.....	23
Panier d'achat.....	25
MySQL.....	28
Accès à MySQL.....	28
Requête SQL.....	29

Node.js est un environnement de code source ouvert qui permet d'exécuter du code JavaScript en dehors du navigateur Web et sur n'importe quelle plateforme. Node.js possède un vaste écosystème de librairies open source disponibles pour les développeurs.

Introduction à Node.js

Votre premier programme node.js

1. Créer un nouveau fichier `boujour.js` et Entrez le code suivant:

```
function saluer(nom) {  
  console.log("Bonjour " + nom);  
}  
  
saluer("le monde!!!");
```

2. Créez une fonction `saluer(nom)` qui affiche son contenu du paramètre à la console.
3. Appelez ensuite la fonction `saluer()` en passant en paramètre la chaîne `"le monde!!!"`
4. Enregistrez le fichier `boujour.js`
5. À la ligne de commande, exécutez la commande `node boujour.js`
6. Le résultat du programme devrait s'afficher sur la console.

Saisir de l'information

1. Créer un nouveau fichier `lireNom.js` et entrez le code suivant :

```
const readline = require("readline");  
const rl = readline.createInterface({  
  input: process.stdin,  
  output: process.stdout  
});  
  
rl.question(`Quel est votre nom? `, (nom) => {  
  console.log(`Hi ${nom}!`)  
  rl.close()  
});
```

2. À la ligne de commande, exécutez la commande `node lireNom.js`

3. Le résultat suivant devrait s'afficher sur la console.

```
Quel est votre nom? Stéphane  
Hi Stéphane!
```

Note : node.js est asynchrone.

1. Créer un nouveau fichier `pouceCm.js` et entrez le code suivant :

```
const readline = require("readline");  
const rl = readline.createInterface({  
  input: process.stdin,  
  output: process.stdout  
});  
  
rl.question(`Entrez une dimension en pouce? `, (pouce) => {  
  let cm = 2.54 * pouce;  
  console.log(`Dimension en cm: ${cm}`);  
  rl.close();  
});
```

2. Enregistrez et exécutez le code.
3. Le résultat suivant devrait s'afficher sur la console.

```
Entrez une dimension en pouce? 12  
Dimension en cm: 30.48
```

Lire un fichier texte

1. Créer un nouveau fichier `nom.txt` qui contiendra votre nom.
2. Créer un nouveau fichier `lireFichier.js`
3. Entrez le code suivant :

```
const fs = require('fs');  
fs.readFile("nom.txt", "utf8", (err, data) => {  
  if (err) {  
    console.error(err);  
    return;  
  }  
  console.log(data);  
});
```

```
});
```

4. Enregistrez et exécutez le code.

Écrire un fichier texte

1. Créer un nouveau fichier `ecrireFichier.js`
2. Entrez le code suivant :

```
const fs = require('fs');
let nom = "Stéphane Gill"
fs.writeFile('nouveauFichier.txt', nom, err => {
  if (err) {
    console.error(err);
    return;
  }
  console.log('Sauvegarder!');
});
```

3. Enregistrez et exécutez le code.

Serveur Web

Parmi les modules de `node.js`, on retrouve `http` qui permet le développement de serveur HTTP. Il est donc possible de se passer de serveurs web tels que Nginx ou Apache lors du déploiement de sites et d'applications web développés avec `node.js`.

Serveur Web simple

1. Créez un fichier `serveurWeb.js`
2. Entrez le code suivant :

```
const http = require("http");
const server = http.createServer();
server.listen(4200);
server.on("connection", socket => {
```

```
console.log("Nouvelle connexion...");
});
```

3. Enregistrez et exécutez le code. Ouvrez votre navigateur web et aller à l'adresse suivante : `http://127.0.0.1:4200`. Observez le résultat sur la console.

Le module `http` doit être importé avant de créer un serveur Web à l'aide de la méthode `createServer()`. Pour connecter le serveur sur un port donné il suffit d'appeler la méthode `listen()` de l'objet `server`. La méthode `listen()` reçoit en paramètre le port d'écoute des requêtes HTTP.

Dès qu'une nouvelle connexion ou une requête est créée, l'objet `server` va déclencher un événement. Il faut utiliser la méthode `on()` de l'objet `server` pour gérer l'événement déclenché. La méthode `on()` reçoit en paramètre le nom de l'événement déclenché et une fonction de rappel avec comme paramètre un objet de type `Socket`.

1. Commenter la ligne de code `server.on('connection', (socket) => {...});`
2. Modifiez le code de l'exemple précédent pour appeler la méthode `createServer()` avec une fonction de rappel qui prend deux paramètres `request` et `response`

```
const server = http.createServer((request, response) => {
  response.write("<h1>Bonjour le monde!!!</h1>");
  response.end();
});
```

3. Enregistrez et exécutez le code. Ouvrez votre navigateur web et aller à l'adresse suivante : `http://localhost:4200`. Observez le résultat dans le navigateur. Ici on demande à l'objet `response` d'afficher *Bonjour le monde* sur la page web et ensuite de terminer la réponse en appelant la méthode `end()`.

Référence: <https://nodejs.org/es/docs/guides/anatomy-of-an-http-transaction/>

Routage

1. Créez un fichier `routage.js` et entrez le code suivant :

```
const http = require("http");
const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.write("<h1>Route 0</h1>");
  }
});
```

```

        res.end();
    }
    else if (req.url === "/route1") {
        res.write("<h1>Route 1</h1>");
        res.end();
    }
    else if (req.url === "/route2") {
        res.write("<h1>Route 2</h1>");
        res.end();
    }
    else {
        res.write("<p>Erreur: Cette page n'existe pas</p>");
        res.end();
    }
});
server.listen(4200);

```

2. Enregistrez et exécutez le code. Ouvrez votre navigateur web et aller aux adresses suivantes:
<http://localhost:4200>,
<http://localhost:4200/route1>, <http://localhost:4200/route2>
et <http://localhost:4200/route3>. Observez les résultats dans le navigateur.

L'objet `req` contient toutes les informations que le navigateur Web a demandées. On y retrouve le nom de la page appelée, les paramètres... L'objet `req` contient la réponse que le serveur va retourner au navigateur Web, en général du code HTML ou du JSON.

Route avec paramètre

1. Créez un fichier `param.js` et entrez le code suivant:

```

const http = require("http");

const server = http.createServer((req, res) => {
    let repImages = /\simages/;

    if (req.url === "/") {
        res.write("<h1>Route 0</h1>");
        res.end();
    }
    else if (repImages.test(req.url)) {
        let cheminTab = req.url.split('/');
        console.log(cheminTab);
        res.write("<h1>Repertoire images</h1>");
        if(cheminTab.length > 2)
            res.write("<p>Le nom du fichier est ${cheminTab[2]}</p>");
        else
            res.write("<p>Il n'y a pas de nom de fichier</p>");
        res.end();
    }
});

```

```

    }
    else if (req.url === "/route2") {
        res.write("<h1>Route 2</h1>");
        res.end();
    }
    else {
        res.write("<p>Erreur: Cette page n'existe pas</p>");
        res.end();
    }
});

server.listen(4200);

```

2. Enregistrez et exécutez le code. Ouvrez votre navigateur web et aller à l'adresse suivante : <http://localhost:4200/images/bonjour>. Observez le résultat dans le navigateur.

Fichier HTML

1. Créez un fichier `ServeurMTML.js` et entrez le code suivant :

```

const http = require("http");
const fs = require('fs');
const path = require('path');

const server = http.createServer((req, res) => {
    if (req.url === "/") {
        console.log(__dirname);
        fs.readFile(path.join(__dirname + '/maPage.html'), "utf8",
            (err, data) => {
                if (err) {
                    throw "C'est l'heure d'une pause! " + err;
                }
                res.writeHead(200, { 'Content-Type': 'text/html' });
                res.write(data);
                res.end();
            });
    }
    else {
        res.write("<p>Erreur: Cette page n'existe pas</p>");
        res.end();
    }
});

server.listen(4200);

```

2. Créez un fichier `maPage.html` et entrez le code suivant :

```
<!DOCTYPE html>
```



```
<html>

  <head>
    <meta charset="utf-8" />
    <title>Ma page Web</title>
  </head>
  <body>
    <h1>Voici ma page Web!!!</h1>
  </body>
</html>
```

3. Enregistrez et exécutez le code. Ouvrez votre navigateur web et allez à l'adresse suivante : <http://localhost:4200>. Observez le résultat dans le navigateur.

Fichiers d'images

1. Créez un fichier `ServeurImage.js` et entrez le code suivant :

```
const http = require("http");
const fs = require('fs');

const server = http.createServer((req, res) => {
  let repImages = /\\/images/;
  if (req.url === "/") {
    res.write("<h1>Route 0</h1>");
    res.end();
  }
  else if (repImages.test(req.url)) {
    let nomFichier = "." + req.url;
    let contentType = 'image/jpg';
    let fichier = fs.readFileSync(nomFichier);
    res.writeHead(200, {'Content-Type': contentType });
    res.end(fichier, 'binary');
  }
  else if (req.url === "/route2") {
    res.write("<h1>Route 2</h1>");
    res.end();
  }
  else {
    res.write("<p>Erreur: Cette page n'existe pas</p>");
    res.end();
  }
});

server.listen(4200);
```

2. Créez un dossier `images` contenant une image `img.jpg`.

3. Enregistrez et exécutez le code. Ouvrez votre navigateur web et allez à l'adresse suivante : `http://localhost:4200/images/img.jpg`. Observez le résultat dans le navigateur.

Note : `===` est une comparaison stricte la valeur et le type doivent être égaux.

Modules et NPM

Créer un module

1. Créez un fichier `logger.js` et entrez le code suivant :

```
function log(message) {  
    console.log(message);  
}  
module.exports.log = log;
```

Pour que la fonction `log()` soit visible à l'extérieur du module, il faut l'exporter. L'instruction `module.exports` permet l'exportation

Note : Il est recommandé d'exposer ou d'exporter uniquement ce qui devrait être public aux autres modules et d'éviter d'exporter les détails d'implémentation d'un module

Importation d'un module

1. Créez un fichier `importation.js` et entrez le code suivant :

```
const logger = require("./logger");  
logger.log('test');
```

Note : Il est fortement recommandé de stocker l'objet retourné par la fonction `require()` dans une constante. Ceci évitera de changer accidentellement le contenu de l'objet dans le code. Pour ce faire, il faut utiliser le mot clé `const` au lieu de `let`

Création d'une classe

1. Créez un fichier `Logger.js`

2. Entrez le code suivant:

```
class Logger{
  constructor(niveau){
    this.niveau = niveau;
  }

  afficherMsg(msg){
    console.log(`Niveau ${this.niveau}: ${msg}`);
  }
}
module.exports = Logger;
```

Utilisation d'une classe

1. Créez un fichier utiliserClasse.js
2. Entrez le code suivant:

```
const Logger = require("./Logger");

let monLog = new Logger(2);
monLog.afficherMsg("Voici un log!!!");
```

Gestionnaire de paquets NPM

npm est le gestionnaire de paquets officiel de Node.js. Il permet de télécharger et d'installer des librairies à partir du dépôt npm. Le dépôt npm est accessible via <http://www.npmjs.com>.

Installation d'une librairie

1. Ouvrez votre navigateur web et aller sur <http://www.npmjs.com>;
2. Sur le champ de saisie *Search packages*, recherchez la librairie *underscore*;
3. Cliquez sur la librairie *underscore*
4. Examinez les informations de la librairie *underscore*. Vous pouvez voir entre autres la commande pour installer la librairie, la dernière version de la librairie, le nombre de téléchargements, etc.
5. Ouvrez la ligne de commande Windows ou un terminal;
6. Allez dans le répertoire de votre projet node.js;
7. Entrez la commande `npm i underscore` pour installer la librairie.

Note : La commande `npm i underscore` est équivalente à `npm install underscore`. L'option `i` étant un raccourci de `install`;

Utilisation d'une librairie

Pour utiliser le module *underscore* il faut utiliser les instructions suivantes :

```
const _ = require('underscore');
```

Note : Par convention, on utilise le caractère `_` (*underscore*) pour référer au module *underscore*...
Humour d'informaticien

Services Web avec Express

Qu'est-ce qu'un service REST?

Les services HTTP peuvent être développées en utilisant REST (*Representational State Transfer*). REST est devenu une convention pour développer des services Web

REST utilise le protocole HTTP pour effectuer les opérations suivantes sur les données :

- Création (*Create*)
- Lecture (*Read*)
- Mise à jour (*Update*)
- Suppression (*Delete*)

Ce sont des opérations nommées CRUD pour *Create*, *Read*, *Update* et *Delete*. Le type de requête HTTP détermine le type d'opération à exécuter. Chaque requête HTTP possède un verbe ou une méthode qui détermine son intention. Voici le standard des méthodes HTTP :

- **GET** (Retourner les données);
- **POST** (Créer les données);
- **PUT** (Mettre à jour les données);
- **DELETE** (Supprimer les données);

Installation d'Express

Express est un cadre d'application pour faciliter le développement des applications Web avec Node.js.

1. Dans votre répertoire de travail, exécutez la commande suivante pour installer Express

```
npm install express
```

Serveur Web

1. Créer un nouveau fichier `serveurWeb.js`.
2. Entrez le code suivant :

```
const express = require('express');
const app = express();
const port = 4200;

app.get('/', (request, response) => {
  response.send('<h1>Bonjour le monde!</h1>');
});

app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});
```

L'instruction `require('express')` (ligne 1) permet de charger le module `express`. L'appel à la fonction `express()`, à la ligne 2, crée une application `express` et retourne un objet de type `express`. Par convention, l'objet retourné est stocké dans une variable nommée `app`.

La fonction `get()` de l'objet `app`, en lui passant deux paramètres, permet définir une route : – L'URL sous forme de chaîne de caractères – La fonction de rappel qui reçoit deux paramètres : `request` et `response`.

La réponse `Bonjour le monde !` est envoyée en utilisant la méthode `send()` de l'objet `response`. La méthode `listen()` de l'objet `app` permet d'écouter sur un port donné. Cette méthode reçoit en paramètre le numéro de port ainsi qu'une fonction de rappel qui sera appelée lorsque le serveur est en écoute.

Note : La documentation sur les objets et méthodes utilisées par Express se trouve sur le site <http://expressjs.com/>

Serveur Web de fichier

1. Dans votre répertoire de travail, créez un dossier (...) /public qui contiendra vos fichiers .html, .css, .js, .png...
2. Créez un nouveau fichier serveurFichier.js.
3. Entrez le code suivant :

```
const express = require('express');
const app = express();
const port = 4200;

app.use(express.static(__dirname+'/public'));

app.get('/', (request, response) => {
  response.send('<h1>Bonjour le monde!</h1>');
});

app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});
```

4. Dans le dossier public, créez un nouveau fichier allo.html et entrez le code suivant :

```
<!DOCTYPE html>
<html>
  <body>
    <p>allo</p>
  </body>
</html>
```

5. Ouvrez votre navigateur Web et aller à l'adresse suivante :
<http://127.0.0.1:4200/allo.html>

Serveur Web avec routage

1. Créez un fichier routage.js.
2. Entrez le code suivant :

```
const express = require('express');
const app = express();
const port = 4200;

app.get('/', (req, res) => {
```

```

    res.send('<h1>Route 0</h1>');
  });

app.get('/route1', (req, res) => {
  res.send('<h1>Route 1</h1>');
});

app.get('/route2', (req, res) => {
  res.send('<h1>Route 2</h1>');
});

app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});

```

3. Ouvrez votre navigateur Web et aller aux adresses suivantes : <http://127.0.0.1:4200/>, <http://127.0.0.1:4200/route1> et <http://127.0.0.1:4200/route2>.

Route avec paramètres

1. Créez un fichier `param.js`.
2. Entrez le code suivant :

```

const express = require('express');
const app = express();
const port = 4200;

app.get('/', (req, res) => {
  res.send('<h1>Route 0</h1>');
});

app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});

app.get('/route1/:annee/:mois', (req, res) => {
  let texte = `<h1>Route 1</h1><p>mois: ${req.params.mois}</p>`;
  res.send(texte);
  console.log(req.params);
  console.log(req.query);
});

```

La route `/route1` reçoit 2 paramètres soit : l'année et le mois. Ouvrez votre navigateur Web et aller à l'adresse suivante : <http://127.0.0.1:4200/route1/2018/02?nom=gill>

Serveur Web répondant en JSON

1. Créez un fichier `jsonOut.js` contenant le code suivant :

```
const express = require('express');
const app = express();
const port = 4200;

app.use(function(req, res, next) {
  res.setHeader("Access-Control-Allow-Origin", "*");
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS, PUT, PATCH, DELETE');
  res.setHeader("Access-Control-Allow-Headers", "Origin, X-Requested-With, Content-Type, Accept");
  res.setHeader('Access-Control-Allow-Credentials', true);
  next();
});

app.get('/', (req, res) => {
  let tableau = ["Element1", "Element2", "Element3"];
  res.send(tableau);
});

app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});
```

La méthode `setHeader` permet d'ajouter un entête à la réponse retournée par le serveur. L'ajout d'un entête est nécessaire lorsque l'on retourne du JSON. Dans la route `/`, un tableau contenant 3 chaînes de caractères est créé avant d'être retourné sous forme de JSON.

2. Ouvrez votre navigateur Web et aller à l'adresse suivante: `http://127.0.0.1:4200`
3. Créez un fichier `lireJSON.html` contenant le code suivant :

```
<!DOCTYPE html>
<script>
  const url = "http://127.0.0.1:4200";

  fetch(url)
    .then(rep => rep.json())
    .then(data => alert(data));
</script>
```


Serveur Web recevant du JSON

1. Créez un fichier `jsonIn.js` contenant le code suivant :

```
const express = require('express');
const app = express();
const port = 4200;

app.use(function(req, res, next) {
  res.setHeader("Access-Control-Allow-Origin", "*");
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS, PUT, PATCH, DELETE');
  res.setHeader("Access-Control-Allow-Headers", "Origin, X-Requested-With, Content-Type, Accept");
  res.setHeader('Access-Control-Allow-Credentials', true);
  next();
});

app.use(express.json());

app.post('/', (req, res) => {
  console.log(req.body);
  console.log("nom: " + req.body.nom);
  res.send("<p>Allo</p>");
});

app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});
```

L'instruction `app.use(express.json())` permet au serveur Web de recevoir du JSON. La réception d'information se fait via des routes POST.

2. Utilisez Postman pour envoyer le JSON suivant:

```
{"nom" : "Gill", "prenom" : "Stephane" }
```

3. Créez un fichier `envoyerJSON.html` contenant le code suivant:

```
<!DOCTYPE html>
<script>
  let data = JSON.stringify({nom : "Gill", prenom : "Stephane" });
  fetch("http://127.0.0.1:4200/", {
    method: 'POST',
    body: data,
    headers: new Headers({'Content-Type': 'application/json'
  })),
  })
}
```

```
.then(reponse => reponse.text())  
  .then(data => document.write(data));  
</script>
```

nodemon

L'application nodemon permet de recharger automatiquement votre programme `node.js` lorsque vous modifiez un fichier du code source.

Exécutez la commande `npm i -g nodemon` pour installer nodemon au niveau global.

Une fois installé, il est possible d'exécuter votre application en utilisant la commande `nodemon`

```
nodemon index.js
```

Mongoose

La librairie mongoose vous fournit une API pour accéder à MongoDB.

Installation

Pour installer la librairie mongoose il faut exécuter la commande suivante:

```
npm i mongoose
```

Connexion

1. Créez le fichier `conn.js`
2. Entrez le code suivant :

```
const mongoose = require("mongoose");  
  
mongoose  
  .connect("mongodb://localhost/demo", { useNewUrlParser: true })  
  .then(() => console.log("Connecté à la BD Mongo..."))  
  .catch(error => console.log("Echec de connexion à la BD Mongo...",  
error));
```

3. Exécutez `conn.js` et le message "Connecté à la BD Mongo. . ." devrait apparaître.

Lors de la connexion, il faut spécifier la collection qui sera utilisé. Dans l'exemple précédent c'est la collection `demo` qui est utilisé. Une collection dans MongoDB est similaire à une table dans une BD relationnelle.

Ajouter un enregistrement

```
const mongoose = require("mongoose");

mongoose
  .connect("mongodb://localhost/demo", {useNewUrlParser: true,
useUnifiedTopology: true })
  .then(() => console.log("Connecté à la BD Mongo..."))
  .catch(error => console.log("Echec de connexion à la BD Mongo...",
error));

const contactSchema = new mongoose.Schema({
  nom: String,
  prenom: String,
  telephone: String,
  email: String
});

const Contact = mongoose.model("Contact", contactSchema);

let unContact = new Contact({
  nom: "Gill",
  prenom: "Stephane",
  telephone: "15143895921",
  email: "Stephane.Gill@CollegeAhuntsic.qc.ca"
});

let resultat = unContact.save();

console.log(resultat);
```

La variable `contactSchema` permet de définir le schéma qui sera utilisé. Un schéma définit la structure d'un document appartenant à une collection dans MongoDB. Un document contient une suite de clé/valeur similaire à un objet JSON.

Vérifier l'ajout avec les commandes suivantes:

```
$ mongo
MongoDB shell version v3.6.3
connecting to: mongodb://127.0.0.1:27017
MongoDB server version: 3.6.3
(...)
```

```

> show dbs
admin
0.000GB
config 0.000GB
demo
0.000GB
local
0.000GB
> use demo
switched to db demo
> show collections
contacts
> db.contacts.find()
{ "_id" : ObjectId("5e6bfd2b9248af44ddb06bde"), "nom" : "Gill", "prenom" :
"Stephane", "telephone" : "15
> exit
bye
$

```

Lire les enregistrements

```

const mongoose = require("mongoose");

mongoose
  .connect("mongodb://localhost/projet1", { useNewUrlParser: true,
useUnifiedTopology: true })
  .then(() => console.log("Connecté à la BD Mongo..."))
  .catch(error => console.log("Echec de connexion à la BD Mongo...",
error));

const contactSchema = new mongoose.Schema({
  nom: String,
  prenom: String,
  telephone: String,
  email: String
});

const Contact = mongoose.model("Contact", contactSchema);

//Contact.find({}, (err, res) => console.log(res) );
Contact.find({})
  .sort({nom: -1})
  .exec()
  .then(res => console.log(res))
  .then(() => mongoose.connection.close())
  .catch(err => console.log(err));

```

Service Web REST

```
const express = require('express');
const mongoose = require("mongoose");

const app = express();
const port = 4200;

app.use(function (req, res, next) {
  res.setHeader("Access-Control-Allow-Origin", "*");
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS, PUT, PATCH, DELETE');
  res.setHeader("Access-Control-Allow-Headers", "Origin, X-Requested-With, Content-Type, Accept");
  res.setHeader('Access-Control-Allow-Credentials', true);
  next();
});

app.use(express.json());

mongoose
  .connect("mongodb://localhost/demorest", { useNewUrlParser: true, useUnifiedTopology: true })
  .then(() => console.log("Connecté à la BD Mongo..."))
  .catch(error => console.log("Echec de connexion à la BD Mongo...", error));

const personneSchema = new mongoose.Schema({
  nom: String,
  prenom: String
});

const PersonneModel = mongoose.model("personne", personneSchema);

// Ajouter un enregistrement dans la DB (CREATE)
app.post('/personne', async (request, response) => {
  console.log("Route POST /personne");
  console.log(request.body);
  try {
    let person = new PersonneModel(request.body);
    let result = await person.save();
    response.send(result);
  }
  catch (error) {
    response.status(500).send(error);
  }
});

// Obtenir la liste des enregistrements contenus dans la DB (READ)
app.get('/personnes', async (request, response) => {
  console.log("Route GET /personnes");
  try {
    let result = await PersonneModel.find().exec();
    response.send(result);
  }
});
```

```

    }
    catch (error) {
      response.status(500).send(error);
    }
  });

  // Obtenir un enregistrement en particulier dans la DB (READ)
  app.get("/personne/:id", async (request, response) => {
    console.log("Route GET /personne/:id");
    try {
      let person = await PersonneModel.findById(request.params.id).exec();
      response.send(person);
    }
    catch (error) {
      response.status(500).send(error);
    }
  });

  // Mettre à jour un enregistrement dans la DB (UPDATE)
  app.put("/personne/:id", async (request, response) => {
    console.log("Route PUT /personne/:id");
    console.log(request.body);
    try {
      let person = await PersonneModel.findById(request.params.id).exec();
      person.set(request.body);
      let result = await person.save();
      response.send(result);
    }
    catch (error) {
      response.status(500).send(error);
    }
  });

  // Effacer un enregistrement (EFFACER)
  app.delete("/personne/:id", async (request, response) => {
    try {
      let result = await PersonneModel.deleteOne({ _id:
request.params.id }).exec();
      response.send(result);
    }
    catch (error) {
      response.status(500).send(error);
    }
  });

  app.listen(port, () => {
    console.log(`Serveur en écoute sur le port ${port}...`);
  });

```

Utiliser Postman pour tester votre Service Web.

Référence: <https://www.thepolyglotdeveloper.com/2019/02/building-rest-api-mongodb-mongoose-nodejs/>

Session

Les sessions sont un moyen simple de stocker des données individuelles pour chaque utilisateur en utilisant un identifiant de session unique. Elles peuvent être utilisées pour faire persister des informations entre plusieurs pages. Les identifiants de session sont normalement envoyés au navigateur via des cookies de session, et l'identifiant est utilisé pour récupérer les données existantes de la session. L'absence d'un identifiant ou d'un cookie de session indique au serveur Web de créer une nouvelle session, et génère ainsi un nouvel identifiant de session.

Les sessions suivent une cinématique simple. Lorsqu'une session est démarrée, Le serveur Web va soit récupérer une session existante en utilisant l'identifiant de session passé (habituellement depuis un cookie de session) ou si aucun identifiant de session n'est passé, il va créer une nouvelle session.

Sécurité et authentification

Afin de voir concrètement comment fonctionnent les sessions, prenons alors un exemple simple :

- imaginons qu'un site possède une section membre où chaque membre devra s'authentifier avant de pouvoir y accéder.
- de plus, il faut s'assurer qu'il s'agisse toujours de ce même membre qui est connecté.

Le site web aura alors une page contenant un formulaire permettant aux visiteurs de se connecter à une section membre.

Exemple 1 : Formulaire d'authentification (login.html):

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Formulaire d'authentification</title>
  </head>
  <body>
    <h1>Formulaire d'authentification</h1>
    <form action="auth" method="POST">
      <input type="text" name="username" placeholder="Username"
required>
      <input type="password" name="password" placeholder="Password"
required>
      <input type="submit">
    </form>
  </body>
</html>
```

Ce formulaire transfère son contenu (nom d'utilisateur et mot de passe) à un serveur Web Node.js (login.js) via la route /auth.

```
const express = require('express');
const session = require('express-session');
const path = require('path');

const app = express();
const port = 4200;

app.use(session({
  secret: 'secret',
  resave: true,
  saveUninitialized: true
}));

app.use(express.urlencoded()); /* Pour recevoir les paramètres du POST */

app.get('/', function (req, res) {
  res.sendFile(path.join(__dirname + '/login.html'));
});

app.post('/auth', function (req, res) {
  console.log(req.body);
  let username = req.body.username;
  let password = req.body.password;
  if (username && password) { /* ligne 25 */
    console.log(`username: ${username}; password ${password}`);
    /* Vérifier dans la base de donnée */
    if (username == "gills") { /* ligne 28 */
      req.session.loggedin = true;
      req.session.username = username;
      res.redirect("/home");
    }
    else {
      res.send("Mauvais mot de passe");
    }
  }
  else {
    res.send("Veuillez entrer un mot de passe");
  }
  res.end();
});

app.get('/home', function(req, res) {
  if (req.session.loggedin) {
    res.send("Bienvenue , " + req.session.username + "!");
  }
  else {
    res.send("Pour voir cette page, il faut être connecté");
  }
  res.end();
});
```



```
app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});
```

À la ligne 25, l'existence des variables username et password est vérifiée. Puis, à la ligne 28, la validité du mot de passe est vérifiée. Par la suite, les informations sur l'utilisateur sont stockées dans les variables de session puis l'utilisateur est redirigé vers la page de la section membre (la route /home).

Une étape indispensable dans toutes les pages de la section membre est de vérifier s'il s'agit bien d'un visiteur enregistré en vérifiant le contenu des variables de session.

Panier d'achat

Une méthode simple pour créer un panier d'achat est d'utiliser les variables de sessions.

Panier
+codeArticle: String[] +qteArticle: int[] +prixArticle: float[]
+ajouterArticle(code,qte,prix): void +supprimerArticle(code): void +getPrixTotal(): float +getArticle(code): boolean

La première étape est de créer une classe pour décrire les items que l'on retrouve dans le panier.

```
module.exports = class Panier{
  constructor(panier){
    this.codeArticle = panier.codeArticle || [];
    this.qteArticle = panier.qteArticle || [];
    this.prixArticle = panier.prixArticle || [];
  }

  getArticle(code) {
    for(let i = 0 ; i <this.codeArticle.length ; i++){
      if (code == this.codeArticle[i]){
        return i;
      }
    }
    return -1;
  }

  ajouterArticle(code, qte, prix)
  {
    let index = this.getArticle(code);
```

```

        if (index == -1) {
            this.codeArticle.push(code);
            this.qteArticle.push(qte);
            this.prixArticle.push(prix);
            //this.articles.push(new Item(code, qte, prix));
        }
        else{
            let qteActuel = this.qteArticle[index];
            this.qteArticle[index] = qteActuel + qte;
        }
    }

    getPrixTotal()
    {
        let total = 0;
        for(let i = 0 ; i < this.codeArticle.length ; i++){
            total += this.qteArticle[i] * this.prixArticle[i];
        }
        return total;
    }

    supprimerArticle(code){
        let index = this.getArticle(code);
        if (index > -1){
            this.codeArticle.splice(index, 1);
            this.qteArticle.splice(index, 1);
            this.prixArticle.splice(index, 1);
        }
    }
}

```

Puis, il faut utiliser les variables de session pour stocker le contenu du panier.

```

const express = require('express');
const session = require('express-session');
const Panier = require("./Panier.js");

const app = express();
const port = 4200;

app.use(session({
    secret: 'secret',
    resave: true,
    saveUninitialized: true
})));

app.use(function(req, res, next) {
    res.setHeader("Access-Control-Allow-Origin", "*");
    res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS,
        PUT, PATCH, DELETE');
    res.setHeader("Access-Control-Allow-Headers", "Origin, X-Requested-With,
        Content-Type, Accept");
    res.setHeader('Access-Control-Allow-Credentials', true);

```

```

    next();
  });

app.get('/getPanier', (req, res) => {
  console.log(`Afficher le contenu du panier`);
  if(req.session.panier){
    let monPanier = new Panier(req.session.panier);
    console.log(monPanier);
    res.send(monPanier);
  }
  else{
    console.log("Le panier est vide");
    res.send({});
  }
  res.end();
});

app.post('/addArticle/:code/:qte/:prix', (req, res) => {
  console.log(req.params.code);
  console.log(req.params.qte);
  console.log(req.params.prix);
  console.log(`Ajouter un item ${req.params.code}`);
  let monPanier;
  if(req.session.panier){
    monPanier = new Panier(req.session.panier);
  }
  else{
    monPanier = new Panier({});
  }

  monPanier.ajouterArticle(req.params.code, parseInt(req.params.qte),
    parseFloat(req.params.prix));
  console.log(monPanier);
  req.session.panier = monPanier;
  res.end();
});

app.delete('/supArticle/:code', (req, res) => {
  console.log(`Supprimer un item ${req.params.code}`);
  if(req.session.panier){
    monPanier = new Panier(req.session.panier);
    monPanier.supprimerArticle(req.params.code)
  }
  req.session.panier = monPanier;
  console.log(monPanier)
  res.end();
});

app.listen(port, () => {
  console.log(`Serveur en écoute sur le port ${port}...`);
});

```

Références

<https://flaviocopes.com/express-sessions/> <https://www.npmjs.com/package/express-session>

<https://github.com/auchenberg/nodejs-shopping-cart/blob/master/routes/index.js>

<https://marcautran.developpez.com/tutoriels/nodejs/panier/>

<https://codeshack.io/basic-login-system-nodejs-express-mysql/>

MySQL

Accès à MySQL

1. Créez la base de données suivante :

```
CREATE DATABASE monBlogue;
USE monBlogue;
CREATE TABLE utilisateurs (
  prenom VARCHAR(32) NOT NULL,
  nom VARCHAR(32) NOT NULL,
  username VARCHAR(32) NOT NULL UNIQUE,
  password VARCHAR(32) NOT NULL
);
INSERT INTO `utilisateurs` (`prenom`, `nom`, `username`, `password`)
VALUES ('Stephane', 'Gill', 'gills', 'qwerty');
```

2. Installez le module mysql

```
npm install mysql
```

3. Créez un fichier dbAcces.js contenant les instructions suivantes :

```
const mysql = require('mysql');

const con = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "monBlogue"
});

con.connect((err) => {
  if (err){
    console.log("Not connected!");
    throw err;
  }
});
```

```
    console.log("Connected!");  
    con.end();  
});
```

Requête SQL

1. Modifiez le code de l'exemple précédent pour ajouter la requête sql suivante:

```
(...)  
con.connect((err) => {  
    if (err){  
        console.log("Not connected!");  
        throw err;  
    }  
  
    console.log("Connected!");  
    let sql = "SELECT * FROM utilisateurs";  
  
    con.query(sql, (err, result) => {  
        if (err){  
            console.log(err.message);  
            throw err;  
        }  
  
        Object.keys(result).forEach((key) => {  
            var row = result[key];  
            console.log(row);  
            console.log(row.nom)  
        });  
    });  
    con.end();  
});
```

La méthode `Object.keys()` renvoie un tableau contenant les noms des propriétés propres à un objet (qui ne sont pas héritées via la chaîne de prototypes) et qui sont énumérables. L'ordre de ce tableau est le même que celui obtenu par une boucle `for...in` (à la différence qu'une boucle `for...in` liste également les propriétés héritées).

```
const object1 = {  
    a: 'somestring',  
    b: 42,  
    c: false  
};  
  
console.log(Object.keys(object1));
```

L'exécution de ce programme générera l'affiche suivant :

```
Array ["a", "b", "c"]
```

L'instruction `for...in` permet d'itérer sur les propriétés énumérables d'un objet qui ne sont pas des symboles. Pour chaque propriété obtenue, on exécute une instruction (ou plusieurs grâce à un bloc d'instructions).

```
const object = {a: 1, b: 2, c: 3};  
for (const property in object) {  
  console.log(`${property}: ${object[property]}`);  
}
```

L'exécution de ce programme générera l'affiche suivant:

```
a: 1  
b: 2  
c: 3
```